

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4522 – Ağ Tabanlı Paralel Dağıtım Sistemleri [A]



PROJE

VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME

GitHub: <https://github.com/rustemio/sql1.git>

Video: https://youtu.be/rA_DXLe4KqI



Rustem TUKHBETOV

21291001

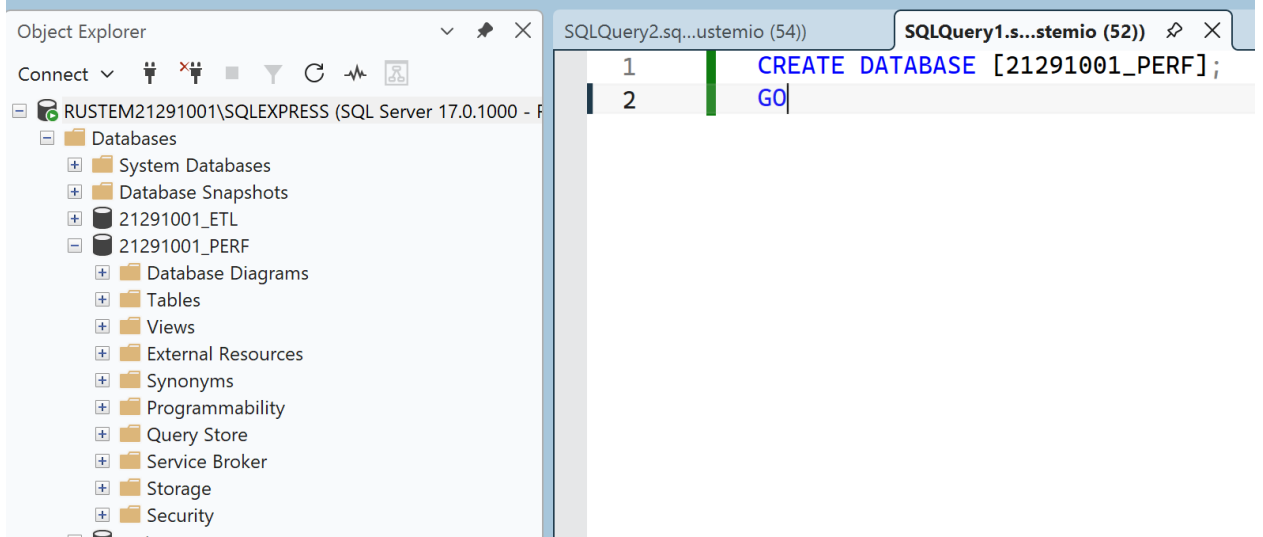
21.04.2026

GİRİŞ VE PROJE AMACI

Projenin temel amacı, Microsoft SQL Server ortamında çalışan veri yapılarının performansını incelemek, yavaş çalışan sorguları tespit etmek ve uygun iyileştirme yöntemleri ile sorgu performansını artırmaktır.

Proje sürecinde örnek veri tabloları üzerinde çeşitli sorgular çalıştırılmış, bu sorguların çalışma davranışları gözlemlenmiş ve performans açısından iyileştirme gerektiren noktalar belirlenmiştir. Daha sonra indeksleme ve sorgu yapısının düzenlenmesi gibi yöntemlerle veritabanı performansını artırmaya yönelik uygulamalar yapılmıştır.

Çalışmanın sonunda sorguların optimizasyon öncesi ve sonrası durumları karşılaştırılmış, performans artışı değerlendirilmiş ve veritabanı izleme yaklaşımı uygulamalı olarak gösterilmiştir.



KULLANILAN ARAÇLAR VE VERİ YAPISI

Bu projede veritabanı performansını incelemek ve iyileştirmek amacıyla Microsoft SQL Server ve SQL Server Management Studio kullanılmıştır. Çalışma sırasında performans karşılaştırmasını daha anlaşılır hale getirmek için müşteri, ürün ve sipariş mantığına uygun örnek tablolar üzerinde çalışılması planlanmıştır.

Performans optimizasyonu projesinde önemli olan nokta yalnızca veri saklamak değil, aynı zamanda sorguların hangi koşullarda yavaşladığını gözlemleyebilmektir. Bu nedenle veri yapısı, filtreleme, sıralama ve birleştirme işlemlerini test etmeye uygun olacak şekilde hazırlanacaktır. Böylece indeksleme ve sorgu iyileştirme adımlarının etkisi daha net biçimde gösterilebilecektir.

```
1 USE [21291001_PERF];
2 GO
3
4 CREATE TABLE customers_perf (
5     customer_id INT PRIMARY KEY,
6     first_name NVARCHAR(100),
7     last_name NVARCHAR(100),
8     city NVARCHAR(100),
9     country NVARCHAR(100),
10    email NVARCHAR(150)
11 );
12
13 CREATE TABLE products_perf (
14     product_id INT PRIMARY KEY,
15     product_name NVARCHAR(200),
16     category NVARCHAR(100),
17     price DECIMAL(10,2)
18 );
19
20 CREATE TABLE orders_perf (
21     order_id INT PRIMARY KEY,
22     customer_id INT,
23     product_id INT,
24     order_date DATE,
25     quantity INT,
```

121 % No issues found

Messages

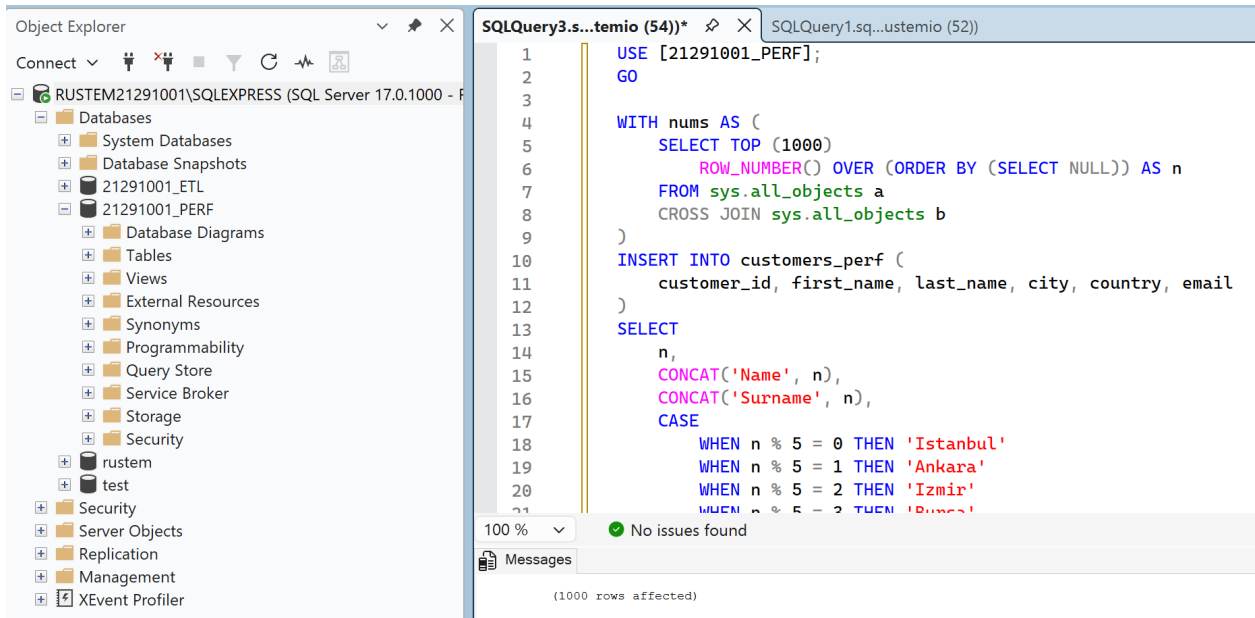
Commands completed successfully.

Completion time: 2026-04-20T19:56:54.1402404+03:00

TEST VERİLERİNİN OLUŞTURULMASI

Performans optimizasyonunu sağlıklı biçimde inceleyebilmek için tabloların belirli bir veri hacmi ile doldurulması gerekmektedir. Bu nedenle proje kapsamında müşteri, ürün ve sipariş tabloları için örnek test verileri oluşturulmuştur. Amaç, sorguların az sayıda kayıt üzerinde değil, daha anlamlı bir veri yoğunluğu altında nasıl davrandığını gözlemlemektir.

Özellikle sipariş tablosu, performans testi açısından en kritik yapı olarak değerlendirilmiştir. Çünkü filtreleme, sıralama, birleştirme ve toplulaştırma işlemleri en yoğun biçimde bu tablo üzerinde yapılacaktır. Böylece indeksleme öncesi ve sonrası fark daha net biçimde ölçülebilecektir.



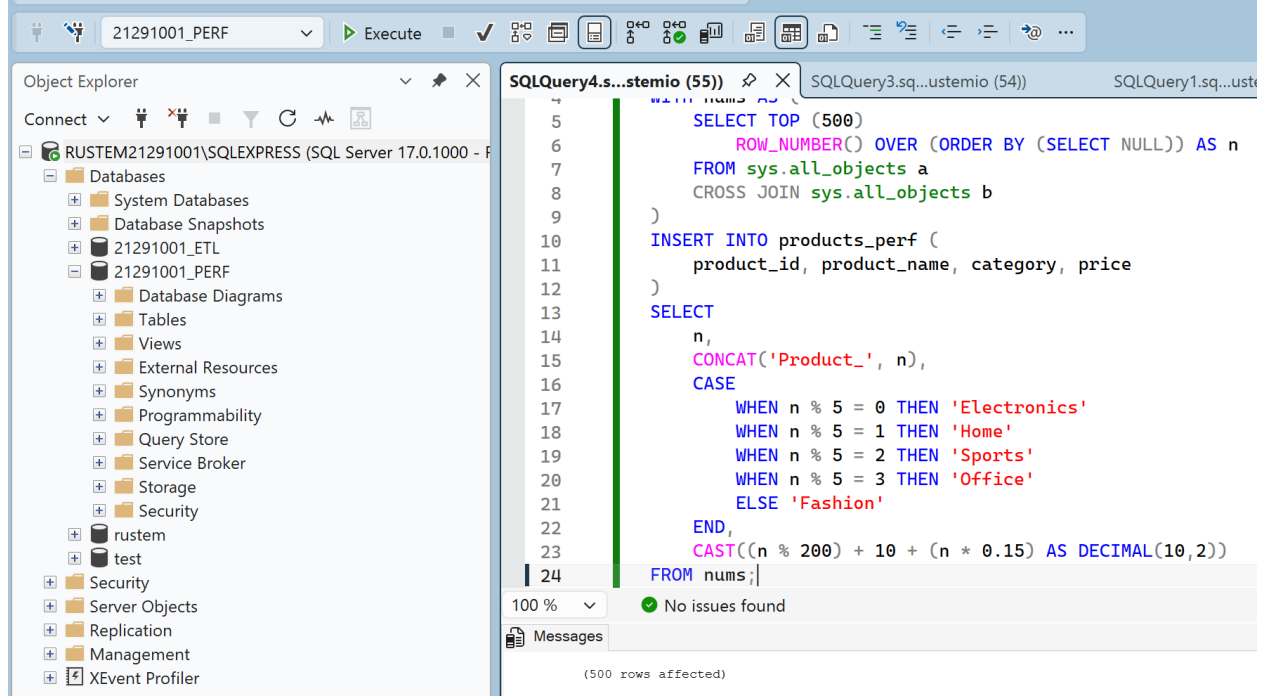
```
1 USE [21291001_PERF];
2 GO
3
4 WITH nums AS (
5     SELECT TOP (1000)
6         ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) AS n
7     FROM sys.all_objects a
8     CROSS JOIN sys.all_objects b
9 )
10 INSERT INTO customers_perf (
11     customer_id, first_name, last_name, city, country, email
12 )
13 SELECT
14     n,
15     CONCAT('Name', n),
16     CONCAT('Surname', n),
17     CASE
18         WHEN n % 5 = 0 THEN 'Istanbul'
19         WHEN n % 5 = 1 THEN 'Ankara'
20         WHEN n % 5 = 2 THEN 'Izmir'
21         WHEN n % 5 = 3 THEN 'Bursa'
```

100 % No issues found

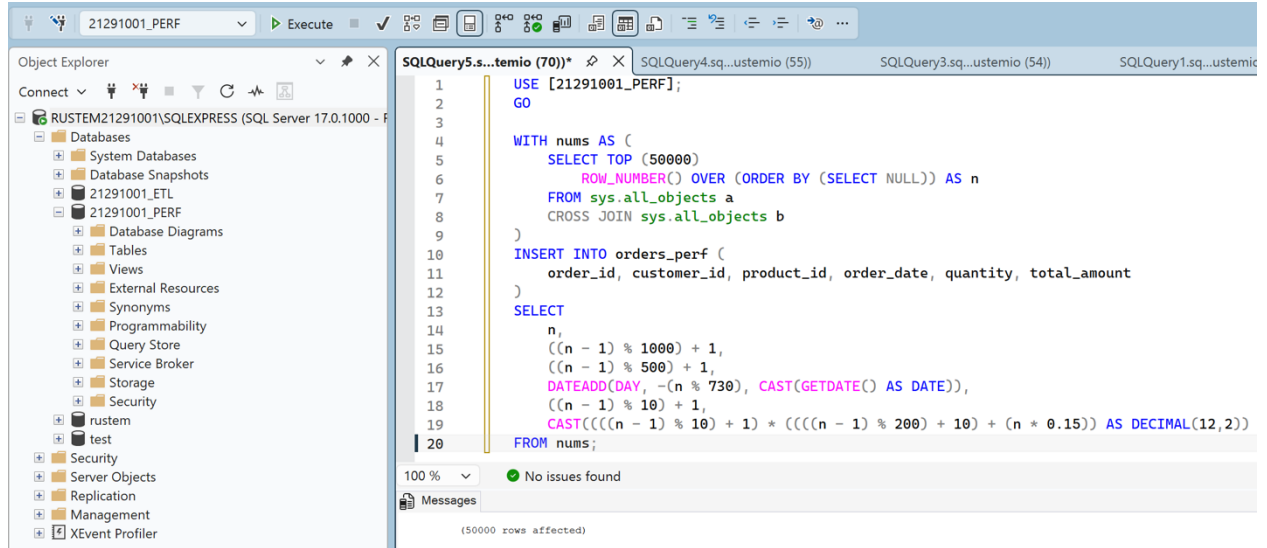
Messages

(1000 rows affected)

Müşteri tablosu doldurulduktan sonra, performans testlerinde kullanılacak ürün verileri de sisteme eklenmiştir. Ürün tablosu, sipariş verileri ile birlikte çalıştırılacak sorgularda birleştirme ve filtreleme işlemlerini desteklemek amacıyla hazırlanmıştır. Böylece proje kapsamında daha gerçekçi bir veri ilişkisi oluşturulmuştur.



Performans analizinin daha anlamlı sonuçlar vermesi için sipariş tablosu diğer tablolara göre daha yüksek sayıda kayıt ile doldurulmuştur. Çünkü performans optimizasyonu açısından en fazla yük oluşturan işlemler genellikle sipariş veya işlem tablosu üzerinde gerçekleşmektedir. Bu nedenle sipariş verisi, proje kapsamında sorgu performansını gözlemlemek için ana çalışma alanı olarak kullanılmıştır.



VERİ HACMİNİN KONTROL EDİLMESİ

Tablolar test verileri ile doldurulduktan sonra, veri hacmi kontrol edilmiştir. Bu aşamanın amacı, performans analizine başlamadan önce her tabloda yeterli sayıda kayıt bulunduğunu doğrulamaktır. Özellikle sipariş tablosundaki kayıt yoğunluğu, sorgu performansını gözlemlemek açısından önemli bir temel oluşturmuştur.

Veri hacminin kontrol edilmesi, proje sürecinde yapılacak karşılaştırmalar için de gereklidir. Çünkü optimizasyon öncesi ve sonrası performans farkını anlamlı biçimde değerlendirebilmek için sorguların belirli bir veri büyüklüğü üzerinde çalışması beklenir.

	customers_count
1	1000
	products_count
1	500
	orders_count
1	50000

Veri hacmi doğrulandıktan sonra, optimizasyon öncesindeki sorgu davranışını gözlemlemek amacıyla ilk performans testleri gerçekleştirilmiştir. Bu aşamada özellikle sipariş tablosu üzerinde filtreleme, sıralama ve birleştirme içeren sorgular çalıştırılmıştır. Amaç, performans açısından iyileştirmeye açık olan sorguları belirlemektir.

İlk test sorguları, veritabanında herhangi bir ek indeksleme uygulanmadan önce çalıştırılmıştır. Böylece başlangıç performansı gözlemlenmiş ve daha sonra yapılacak optimizasyon işlemlerinin etkisi karşılaştırmalı olarak değerlendirilebilecek hale gelmiştir.

İLK PERFORMANS SORGULARININ ÇALIŞTIRILMASI

İlk performans testi sonucunda sorgunun optimizasyon öncesindeki çalışma davranışı gözlemlenmiştir. Sorgu sonucunda elapsed time, CPU time ve logical read değerleri incelenmiştir. Özellikle orders_perf ve customers_perf tabloları üzerinde oluşan okuma sayıları, sorgunun veri erişim maliyetini değerlendirmek açısından önemli bir gösterge sağlamıştır.

Bu ilk ölçüm, optimizasyon çalışmasının başlangıç noktası olarak kabul edilmiştir. Daha sonra uygulanacak indeksleme ve sorgu iyileştirme adımları ile aynı sorgu tekrar çalıştırılacak ve elde edilen sonuçlar karşılaştırılacaktır.

```
1 USE [21291001_PERF];
2 GO
3 SET STATISTICS TIME ON;
4 SET STATISTICS IO ON;
5 GO
6
7 SELECT TOP 1000
8     o.order_id,
9     o.order_date,
10    o.quantity,
11    o.total_amount,
12    c.first_name,
13    c.last_name,
14    p.product_name,
15    p.category|
```

133 % No issues found

Results Messages

SQL Server Execution Times:
CPU time = 0 ms, elapsed time = 1 ms.

SQL Server parse and compile time:
CPU time = 187 ms, elapsed time = 256 ms.

(1000 rows affected)

Table 'customers_perf'. Scan count 0, logical read
Table 'Worktable'. Scan count 0, logical reads 0,
Table 'Workfile'. Scan count 0, logical reads 0, p
Table 'orders_perf'. Scan count 1, logical reads 2
Table 'products_perf'. Scan count 1, logical reads

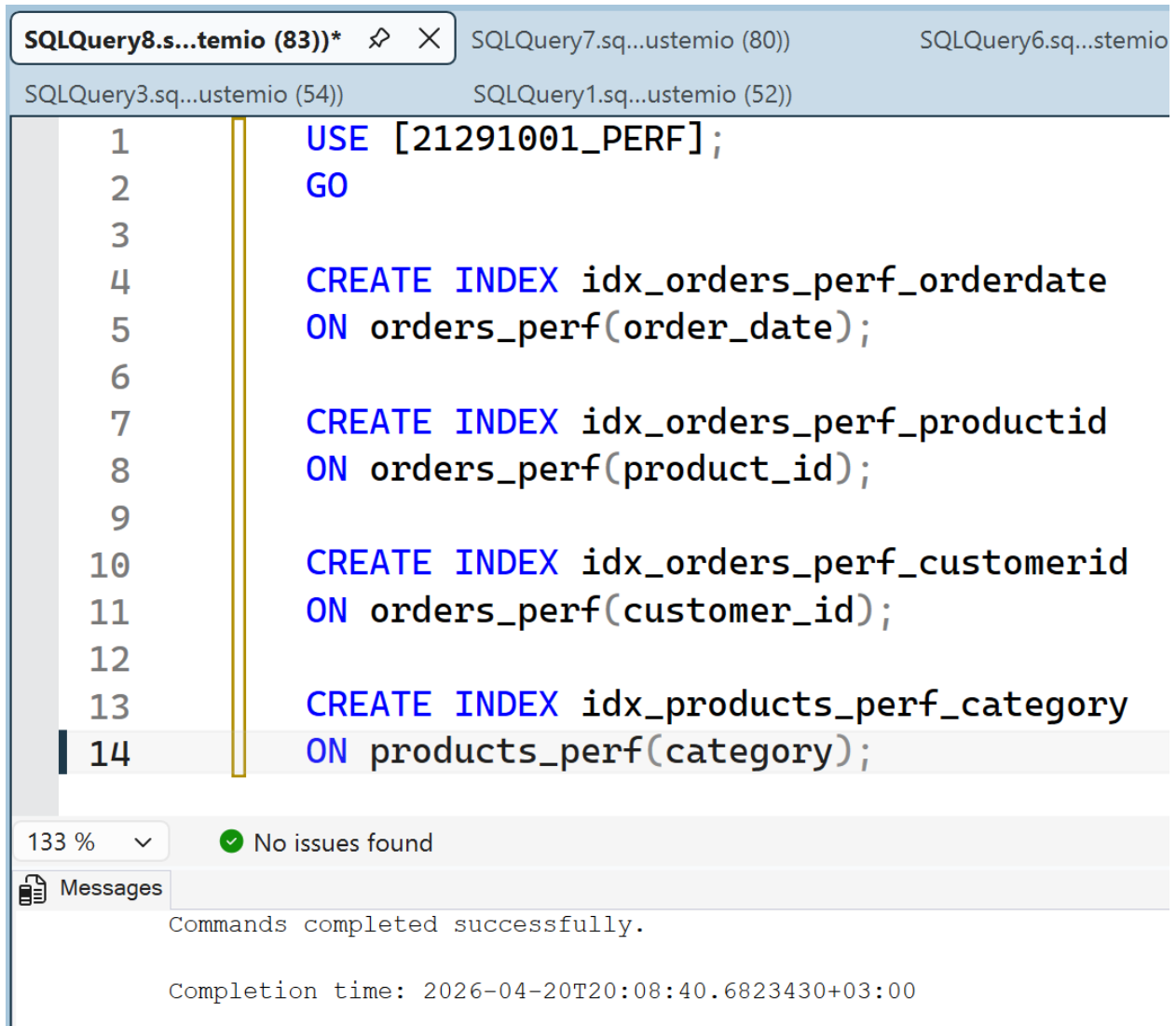
SQL Server Execution Times:
CPU time = 32 ms, elapsed time = 153 ms.

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

İNDEKSLEME İLE OPTİMİZASYON

İlk performans ölçümünden sonra sorgunun daha verimli çalışabilmesi için uygun indeksleme adımı planlanmıştır. Performans testinde özellikle orders_perf tablosundaki tarih alanı ve ürün ile müşteri ilişkilerini içeren alanlar ön plana çıkmıştır. Bu nedenle sorguda filtreleme ve birleştirme işlemlerinde kullanılan sütunlar üzerinde indeks oluşturulması uygun görülmüştür.

İndeksleme işleminin amacı, SQL Server'ın ilgili kayıtlara daha hızlı ulaşmasını sağlamak ve gereksiz tarama maliyetini azaltmaktır. Böylece aynı sorgunun optimizasyon sonrasında daha düşük okuma maliyeti ve daha kısa çalışma süresi ile sonuç vermesi beklenmektedir.



The screenshot displays the SQL Server Enterprise Manager interface. At the top, there are tabs for various SQL queries: 'SQLQuery8.s...temio (83))*', 'SQLQuery7.sq...ustemio (80))', 'SQLQuery6.sq...stemio', 'SQLQuery3.sq...ustemio (54))', and 'SQLQuery1.sq...ustemio (52))'. The main window shows a script editor with the following SQL commands:

```
1 USE [21291001_PERF];
2 GO
3
4 CREATE INDEX idx_orders_perf_orderdate
5 ON orders_perf(order_date);
6
7 CREATE INDEX idx_orders_perf_productid
8 ON orders_perf(product_id);
9
10 CREATE INDEX idx_orders_perf_customerid
11 ON orders_perf(customer_id);
12
13 CREATE INDEX idx_products_perf_category
14 ON products_perf(category);
```

Below the script editor, there is a status bar showing '133 %' and a green checkmark with the text 'No issues found'. A 'Messages' pane at the bottom shows the message 'Commands completed successfully.' and the completion time 'Completion time: 2026-04-20T20:08:40.6823430+03:00'.

OPTİMİZASYON SONRASI PERFORMANS TESTİ

İndeksleme işlemi tamamlandıktan sonra aynı sorgu tekrar çalıştırılmış ve optimizasyon öncesi ölçümler ile yeni sonuçlar karşılaştırılmıştır. Bu aşamadaki amaç, oluşturulan indekslerin sorgu performansına olan etkisini somut biçimde göstermektir. Özellikle elapsed time, CPU time ve logical read değerleri yeniden incelenerek performans artışı değerlendirilmiştir.

Bu karşılaştırma, veritabanı performans optimizasyonu çalışmasının en önemli bölümünü oluşturmaktadır. Çünkü aynı sorgunun önce ve sonra elde ettiği sonuçlar, uygulanan iyileştirmenin gerçekten etkili olup olmadığını açık biçimde göstermektedir.

```

1      USE [21291001_PERF];
2      GO
3      SET STATISTICS TIME ON;
4      SET STATISTICS IO ON;
5      GO
6
7      SELECT TOP 1000
8          o.order_id,
9          o.order_date,
10         o.quantity,
11         o.total_amount,
12         c.first_name,
13         c.last_name,
14         p.product_name,
15         p.category

```

133 % No issues found

Results Messages

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, physical writes 0, scan time 0.000 seconds, elapsed time 0.000 seconds, SQL Server execution time 0.000 seconds.

Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, physical writes 0, scan time 0.000 seconds, elapsed time 0.000 seconds, SQL Server execution time 0.000 seconds.

Table 'orders_perf'. Scan count 1, logical reads 232, physical reads 0, physical writes 0, scan time 0.000 seconds, elapsed time 0.000 seconds, SQL Server execution time 0.000 seconds.

Table 'products_perf'. Scan count 1, logical reads 6, physical reads 0, physical writes 0, scan time 0.000 seconds, elapsed time 0.000 seconds, SQL Server execution time 0.000 seconds.

SQL Server Execution Times:
 CPU time = 62 ms, elapsed time = 134 ms.

SQL Server parse and compile time:
 CPU time = 0 ms, elapsed time = 0 ms.

Completion time: 2026-04-20T20:11:25.6667953+03:00

DMV İLE VERİTABANI İZLEME

Performans optimizasyonu çalışmasında yalnızca sorgu sürelerinin karşılaştırılması yeterli değildir. Aynı zamanda SQL Server ortamında sorguların sistem üzerindeki etkisini izlemek de önemlidir. Bu nedenle proje kapsamında Dynamic Management Views (DMV) yapısından yararlanılarak en fazla kaynak kullanan sorgular incelenmiştir.

DMV sorguları sayesinde SQL Server üzerinde çalışan sorguların toplam işlem süresi, mantıksal okuma miktarı ve son çalışma zamanı gibi bilgiler gözlemlenebilmektedir. Bu yaklaşım, performans problemlerinin yalnızca tahmine dayalı değil, ölçülebilir sistem verileri üzerinden değerlendirilmesini sağlamaktadır.

```
1  USE [21291001_PERF];
2  GO
3
4  SELECT TOP 10
5      qs.execution_count,
6      qs.total_worker_time / 1000 AS total_cpu_ms,
7      qs.total_elapsed_time / 1000 AS total_elapsed_ms,
8      qs.total_logical_reads,
9      qs.last_execution_time,
10     SUBSTRING(
11         st.text,
12         (qs.statement_start_offset / 2) + 1,
13         (
14             (CASE qs.statement_end_offset
15              WHEN -1 THEN DATALENGTH(st.text)
```

133 % No issues found

Results Messages

	execution_count	total_cpu_ms	total_elapsed_...	total_logical_reads	last_exec...	query_text
1	4	391	506	107	2026-04-...	SELECT target_data
2	1	387	387	87592	2026-04-...	SELECT SCHEMA_NAME
3	5	225	315	3638	2026-04-...	SELECT dtb.name AS [Nai
4	1	264	269	3086	2026-04-...	WITH nums AS (SELEC
5	1	242	242	121662	2026-04-...	SELECT SCHEMA_NAME
6	1	175	176	118	2026-04-...	SELECT udf.name AS [Nar
7	1	128	139	85417	2026-04-...	SELECT SCHEMA_NAME
8	1	44	121	2238	2026-04-...	SELECT TOP 1000 o.or
9	2	85	85	4080	2026-04-...	SELECT i.name AS [Name
10	2	73	73	489	2026-04-...	SELECT tbl.name AS [Nan

VERİ YÖNETİCİSİ ROLLERİ VE ERİŞİM YÖNETİMİ

Proje kapsamında veritabanı performans optimizasyonuna ek olarak temel düzeyde erişim yönetimi de uygulanmıştır. Farklı kullanıcı ihtiyaçlarını temsil etmek amacıyla farklı roller tanımlanmış ve bu rollere uygun yetkiler verilmiştir. Böylece veritabanı üzerinde yalnızca performans değil, güvenli ve kontrollü erişim yapısı da gösterilmiştir.

Bu çalışma kapsamında okuma amaçlı kullanıcılar, raporlama kullanıcıları ve daha geniş yetkiye sahip kullanıcılar için örnek rol yapıları hazırlanmıştır. Böylece aynı veritabanı içinde farklı erişim seviyelerinin nasıl yönetilebileceği uygulamalı olarak gösterilmiştir.

	name	type_desc
1	readonly_role	DATABASE_ROLE
2	reporting_role	DATABASE_ROLE
3	manager_role	DATABASE_ROLE

Özel roller oluşturulduktan sonra, bu rollerin gerçekten nasıl çalıştığını göstermek için örnek kullanıcılar ilgili rollere atanmıştır. Böylece yetkilerin yalnızca tanımlanmadığı, aynı zamanda uygulamada nasıl davrandığının da test edilmesi mümkün hale gelmiştir.

```
1  USE [21291001_PERF];
2  GO
3
4  GRANT SELECT ON customers_perf TO readonly_role;
5
6  GRANT SELECT ON customers_perf TO reporting_role;
7  GRANT SELECT ON products_perf TO reporting_role;
8  GRANT SELECT ON orders_perf TO reporting_role;
9
10 GRANT SELECT, INSERT, UPDATE, DELETE ON customers_perf TO manager_role;
11 GRANT SELECT, INSERT, UPDATE, DELETE ON products_perf TO manager_role;
12 GRANT SELECT, INSERT, UPDATE, DELETE ON orders_perf TO manager_role;
13 GO
```

133 % No issues found

Messages

Commands completed successfully.

İlk olarak yalnızca sınırlı okuma yetkisine sahip kullanıcı test edilmiştir. Bu testte kullanıcı bağlamı geçici olarak değiştirilmiş ve kullanıcının hangi tabloya erişebildiği kontrol edilmiştir. Böylece readonly_role rolünün gerçekten sınırlı erişim mantığı ile çalıştığı uygulamalı olarak gösterilmiştir.

```
1  USE [21291001_PERF];
2  GO
3
4  EXECUTE AS USER = 'user_readonly';
5  GO
6
7  SELECT TOP 5 * FROM customers_perf;
8  GO
9
10 REVERT;
11 GO
```

133 % No issues found

	customer_id	first_name	last_name	city	country	email
1	1	Name1	Surname1	Ankara	Germany	customer1@mail.com
2	2	Name2	Surname2	Izmir	France	customer2@mail.com
3	3	Name3	Surname3	Bursa	Turkey	customer3@mail.com
4	4	Name4	Surname4	Antalya	Germany	customer4@mail.com
5	5	Name5	Surname5	Istanbul	France	customer5@mail.com

Burada readonly_role rolüne bağlı kullanıcıyı yetkisiz bir tablo üzerinde test ediyorum. Kullanıcı customers_perf tablosunu okuyabiliyordu, ancak products_perf tablosuna erişmeye çalışıldığında yetki hatası alıyor. Bu da rolün gerçekten sınırlı erişim ile çalıştığını göstermektedir.

```
1  USE [21291001_PERF];
2  GO
3
4  EXECUTE AS USER = 'user_readonly';
5  GO
6
7  SELECT TOP 5 * FROM products_perf;
8  GO
9
10 REVERT;
11 GO
```

133 % No issues found

Messages

Msg 229, Level 14, State 5, Line 7
The SELECT permission was denied on the object 'products_perf'

Bir sonraki aşamada reporting rolüne bağlı kullanıcı test edilmiştir. Bu rol, raporlama amaçlı tasarlandığı için temel tablolar üzerinde okuma yetkisine sahiptir. Ancak veri değiştirme yetkisi bulunmamaktadır. Bu nedenle test sırasında hem erişebildiği işlemler hem de erişemediği işlemler ayrı ayrı kontrol edilmiştir.

```
1      USE [21291001_PERF];
2      GO
3
4      EXECUTE AS USER = 'user_reporting';
5      GO
6
7      SELECT TOP 5 * FROM customers_perf;
8      SELECT TOP 5 * FROM products_perf;
9      SELECT TOP 5 * FROM orders_perf;
10     GO
11
12     REVERT;
13     GO
```

133 % No issues found

Results Messages

	customer_id	first_name	last_name	city	country	email
1	1	Name1	Surname1	Ankara	Germany	customer1@mail.com
2	2	Name2	Surname2	Izmir	France	customer2@mail.com
3	3	Name3	Surname3	Bursa	Turkey	customer3@mail.com
4	4	Name4	Surname4	Antalya	Germany	customer4@mail.com
5	5	Name5	Surname5	Istanbul	France	customer5@mail.com

	product_id	product_name	category	price
1	1	Product_1	Home	11.15
2	2	Product_2	Sports	12.30
3	3	Product_3	Office	13.45
4	4	Product_4	Fashion	14.60
5	5	Product_5	Electronics	15.75

	order_id	customer_id	product_id	order_date	quantity	total_amount
1	1	1	1	2026-04-19	1	10.15
2	2	2	2	2026-04-18	2	22.60
3	3	3	3	2026-04-17	3	37.35
4	4	4	4	2026-04-16	4	54.40
5	5	5	5	2026-04-15	5	73.75

Reporting rolüne bağlı kullanıcı tüm ana tablolar üzerinde okuma işlemi gerçekleştirebilmiştir. Ancak bu rol veri güncelleme yetkisine sahip değildir. Bu nedenle erişim yönetimi testinin ikinci aşamasında kullanıcının veri değiştirme girişimi ayrıca kontrol edilmiştir. Beklenen sonuç, güncelleme işleminin yetki hatası ile engellenmesidir.

Msg 229, Level 14, State 5, Line 7

The UPDATE permission was denied on the object 'customers_perf'.

Son aşamada manager rolüne bağlı kullanıcı test edilmiştir. Bu rol, diğer rollere göre daha geniş yetkilere sahiptir ve veri üzerinde okuma, ekleme, güncelleme ve silme işlemleri yapabilecek şekilde tanımlanmıştır. Bu nedenle test sırasında veri güncelleme işlemi uygulanmış ve rolün yetki düzeyi uygulamalı olarak doğrulanmıştır.

```
1      USE [21291001_PERF];
2      GO
3
4      EXECUTE AS USER = 'user_manager';
5      GO
6
7      UPDATE customers_perf
8      SET city = 'ManagerTest'
9      WHERE customer_id = 1;
10     GO
11
12     SELECT TOP 1 *
13     FROM customers_perf
14     WHERE customer id = 1;
```

133 % No issues found

Results Messages

customer_id	first_name	last_name	city	country	email
1	Name1	Surname1	ManagerTest	Germany	customer1@mail.com

Burada manager_role rolüne bağlı kullanıcıyı test ediyorum. Bu kullanıcı veri üzerinde güncelleme işlemi yapabiliyor. Güncelleme sonrasında aynı kaydı tekrar sorguladığımda değişikliğin tabloya işlendiğini görüyorum. Bu da manager rolünün daha geniş erişim yetkisine sahip olduğunu göstermektedir.

(1 row affected)

Manager rolünün veri güncelleme işlemini gerçekleştirebildiği doğrulandıktan sonra, test amacıyla yapılan değişiklik geri alınmıştır. Böylece veri bütünlüğü korunmuş ve gösterim amaçlı işlem kontrollü biçimde tamamlanmıştır.

SONUÇ VE DEĞERLENDİRME

Bu projede Microsoft SQL Server ortamında veritabanı performans optimizasyonu ve izleme süreci uygulamalı olarak gerçekleştirilmiştir. Öncelikle test amacıyla müşteri, ürün ve sipariş tabloları oluşturulmuş ve bu tablolar yeterli veri hacmi ile doldurulmuştur. Daha sonra filtreleme, sıralama ve birleştirme işlemleri içeren sorgular çalıştırılarak optimizasyon öncesi performans ölçümleri alınmıştır.

İlk ölçümlerin ardından sorguda kullanılan alanlar üzerinde indeksleme uygulanmış ve aynı sorgu tekrar çalıştırılarak sonuçlar karşılaştırılmıştır. Karşılaştırma sonucunda sorgu performansında iyileşme gözlemlenmiş ve indeks kullanımının veritabanı performansı üzerindeki etkisi uygulamalı olarak gösterilmiştir. Ayrıca DMV yapıları kullanılarak sistem üzerinde çalışan sorguların izlenmesi sağlanmış ve performans analizi yalnızca sorgu süresi ile değil, sistem verileri ile de desteklenmiştir.

Proje kapsamında erişim yönetimi konusu da temel düzeyde ele alınmıştır. Farklı roller oluşturulmuş, bu rollere uygun kullanıcılar atanmış ve her rolün erişim seviyesi uygulamalı olarak test edilmiştir. Böylece yalnızca performans optimizasyonu değil, aynı zamanda kontrollü erişim yapısı da proje içinde gösterilmiştir.

Sonuç olarak bu çalışma, veritabanı performansını artırmak için doğru indeksleme, sorgu analizi ve sistem izleme yöntemlerinin önemli olduğunu göstermiştir. Aynı zamanda farklı kullanıcı rolleri ile erişim yetkilerinin yönetilmesi sayesinde veritabanı yapısının daha düzenli ve daha kontrollü biçimde kullanılabileceği ortaya konmuştur.